LAB REPORT

Fundamentals of Operating Systems (OS) Assignment – 1

Name : Dhruv Gupta

Roll No : 2401201015

Course : BCA (AI & DS) [H]

Section : A

---

1. Introduction

CPU Scheduling is a fundamental concept in Operating Systems that determines the order in which processes are executed by the CPU. Efficient scheduling ensures optimal utilization of CPU resources and minimizes waiting time for processes.

The two jobs to be implemented :

- First Come First Serve (FCFS)
- Shortest Job First (SJF) – Non-Preemptive

The objective is to understand how different scheduling strategies affect process execution and system performance using parameters such as Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

2. Objectives

- To understand the concept of CPU Scheduling
- To implement FCFS and SJF scheduling algorithms using Python
- To calculate scheduling parameters (CT, TAT, WT)
- To visualize execution using Gantt Charts
- To compare performance of both algorithms

3. Tools and Technologies Used

- Programming Language: Python 3.x
- Code Editor: Visual Studio Code
- Platform: Windows
- Libraries Used: Built-in Python functions

4. Process Structure and Input Handling

4.1 Process Class Design

Each process is represented using a class named Process, which contains the following attributes:

- Process ID (PID)
- Arrival Time (AT)
- Burst Time (BT)
- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)

This structure ensures organized storage and easy computation of scheduling parameters.

```python
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid      # Process ID
        self.at = at        # Arrival Time
        self.bt = bt        # Burst Time
        self.ct = 0         # Completion Time
        self.tat = 0        # Turnaround Time
        self.wt = 0         # Waiting Time
```

0.0s

4.2 Input Handling

The program takes input for multiple processes from the user. Each process includes:

- Arrival Time
- Burst Time

The processes are stored in a list for further scheduling operations.

```
8
Burst Time: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```python
processes = []
n = int(input("Enter number of processes: "))
for i in range(n):
    print(f"\nEnter details for Process {i+1}")

    at = int(input("Arrival Time: "))
    bt = int(input("Burst Time: "))

    processes.append(Process(i+1, at, bt))
```

✓ 36.6s

```python
print("\nPID\tAT\tBT")
for p in processes:
    print(f"P{p.pid}\t{p.at}\t{p.bt}")
```

✓ 0.0s

```
PID     AT      BT
P1      0       7
P2      2       4
P3      4       1
P4      5       4
P5      6       2
```

5. FCFS Scheduling Algorithm

5.1 Working Principle

The First Come First Serve (FCFS) algorithm executes processes in the order of their arrival time. It is a non-preemptive scheduling algorithm.

5.2 Implementation Logic

- Processes are sorted based on Arrival Time
- CPU executes processes sequentially
- If CPU is idle, time is adjusted to next arrival

5.3 Formula Used

- Completion Time (CT) = Previous Time + Burst Time
- Turnaround Time (TAT) = CT – AT
- Waiting Time (WT) = TAT – BT

5.4 Output

The output displays a table containing:

- PID, AT, BT, CT, TAT, WT

```
FCFS Scheduling:
PID     AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P2      2       4       11      9       5
P3      4       1       12      8       7
P4      5       4       16      11      7
P5      6       2       18      12      10
```

6. SJF Scheduling Algorithm (Non-Preemptive)

6.1 Working Principle

Shortest Job First selects the process with the smallest burst time among the processes that have already arrived.

6.2 Implementation Logic

- Maintain a ready queue
- Add processes only when they arrive
- Select process with minimum burst time
- Execute until completion
- Handle CPU idle condition

6.3 Key Features

- Dynamic process selection
- Minimizes waiting time
- Requires knowledge of burst time

6.4 Formula Used

Same as FCFS:

- TAT = CT – AT
- WT = TAT – BT

```
SJF Scheduling (Non-Preemptive):
PID     AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P3      4       1       8       4       3
P5      6       2       10      4       2
P2      2       4       14      12      8
P4      5       4       18      13      9
```

7. Gantt Chart Representation

7.1 Purpose

A Gantt chart visually represents the order and duration of process execution.

7.2 Implementation

From your code (page 4):

- Displays execution timeline
- Shows process sequence
- Indicates completion points

```
Gantt Chart (FCFS):
0 | P1 | 7 | P2 | 11 | P3 | 12 | P4 | 16 | P5 | 18

Gantt Chart (SJF):
0 | P1 | 7 | P3 | 8 | P5 | 10 | P2 | 14 | P4 | 18
```

8. Performance Analysis

8.1 Metrics Used

- Average Waiting Time
- Average Turnaround Time

8.2 Observations

The results show that:

- FCFS follows arrival order strictly
- SJF prioritizes shorter processes
- SJF generally reduces waiting time

```
Average WT (FCFS): 5.8
Average TAT (FCFS): 9.4
```

```
Average Waiting Time (SJF): 4.4
Average Turnaround Time (SJF): 8.0
```

9. Comparison Between FCFS and SJF

Parameter	FCFS	SJF
Scheduling Type	Non-preemptive	Non-preemptive
Selection Basis	Arrival Time	Burst Time
Efficiency	Lower	Higher
Waiting Time	Higher	Lower
Complexity	Simple	Moderate

10. Conclusion

From the analysis, it can be concluded that:

- FCFS is simple but inefficient in many cases
- SJF provides better performance by minimizing waiting time
- SJF is more optimized but requires prior knowledge of burst time
- The choice of scheduling algorithm significantly impacts system performance

11. Challenges Faced

- Handling CPU idle condition
- Implementing correct SJF logic
- Avoiding errors due to shared object references
- Ensuring accurate calculation of CT, TAT, WT

12. Learning Outcomes

- Gained understanding of scheduling algorithms
- Learned implementation of FCFS and SJF

- Understood performance metrics and comparisons
- Improved problem-solving and coding skills

13. GitHub Repository Link

👉 Github Link